PROJECT ASSESSMENT · 23 OCTOBER 2026

Stock TradingDashboard

Detailed project report for an educational Streamlit application featuring market data, intraday charts, and local paper trading.

Important: This is a demonstration application. It does not place real orders, and its simulated stop-loss is not a reliable live-trading mechanism. Do not use it for investment decisions.

Prepared from the project files and local SQLite schema. No credentials or private environment values were inspected or included.

Project type: Python / Streamlit · Persistence: SQLite · Market data: Yahoo Finance via yfinance

1. Executive summary

The project is a compact stock-trading dashboard for educational use. It displays a watchlist of NSE-listed companies, obtains available quotes and intraday candles through Yahoo Finance, and lets a user record simulated buys, update stop-loss levels, and simulate exits.

Application state for trades and holdings is persisted in a local SQLite database. The dashboard is built as a single Streamlit application, while three separate scripts demonstrate Groww API access for account profile, holdings, and quote retrieval.

Overall assessment: A useful small-scale paper-trading demo with persistent trade history and clear user-facing disclaimers. It is not a broker-connected trading system, a reliable real-time data terminal, or a production-ready financial application.

Area	Current state	Assessment
Dashboard	Streamlit tabs for watchlist, selected share, and holdings	Core demo workflow is present
Market data	Yahoo Finance through yfinance	Best-effort; delay, throttling, and gaps possible
Trading	Simulated buys and sells only	No real broker order placement
Persistence	Local SQLite database	Simple, suitable for local single-user demo
Tests	Groww scripts are named with a test_ prefix	Not a dashboard test suite; scripts call external API
Setup	Requirements cover dashboard packages	Groww script dependencies are not listed there

Project strengths

- Clear separation between simulated transactions and real brokerage activity.
- Persistent holdings and event history using a small local database.
- Usable charting workflow, including optional five-minute OHLC resampling.
- Visible warnings about the limitations of market data and stop-loss behavior.

2. Architecture and repository contents

Application flow

1. Streamlit initializes the page and a fixed list of NSE symbols.
2. SQLite tables are created if absent; legacy Streamlit session data is migrated once.
3. The dashboard fetches quotes and intraday history using yfinance.
4. Streamlit renders watchlist metrics, candlestick charts, and pap

er-trading forms.
5 Buy and exit events are recorded in SQLite; holdings are reloade
d from persistent state.
6 On page reruns, open holdings are checked against the latest ava
ilable quote for the demo stop-loss behavior.
Files and responsibilities
File
Purpose
app.py
Streamlit UI, Yahoo Finance access, charting, trade simulation, SQLite initializ
ation and persistence.
README.md
Project overview, setup, usage steps, and free-data / simulation limitations.
requirements.txt
Minimum versions for Streamlit, yfinance, Plotly, and pandas.
test_groww.py
Loads local environment credentials and fetches Groww user profile.
test_groww_holdings.py
Fetches and prints Groww holdings, with error messaging.
test_groww_price.py
Fetches and prints a Groww NSE cash-market quote.
trading_history.db
Local SQLite store containing holdings, trades, and migration metadata tables.
.env
Local environment file exists; contents intentionally excluded from review.
Technology stack
Python, Streamlit, pandas, Plotly, yfinance, and SQLite form the dashboard stack
. The separate Groww scripts also import python-dotenv and growwapi. The project
requirements file currently specifies only the dashboard packages.
3. Features and data model
Market dashboard
The sidebar offers a predefined watchlist and a custom symbol input. The dashboa
rd shows the latest available price and change when data is available. Quote ret
rieval has a daily-history fallback if the provider's fast quote does not contai
n a price.
Selected-share chart
The chart uses one-minute intraday history and can resample it into five-minute
OHLC candles. It displays open, high, low, and close values in a dark-themed Plo
tly candlestick chart. Data availability depends on the upstream provider and ma
rket hours.
Paper trading workflow
- Buy form accepts a positive quantity, buy price, and stop-loss b
elow the buy price.
- Holdings view calculates simple profit/loss as (latest price − b
uy price) × quantity.
- Users can update the stop-loss or record a simulated manual sale
.
- Open positions are checked for stop-loss condition during a Stre
amlit rerun / refresh.
Behavioral limitation: Refresh-triggered checking is not continuous monitoring.
A stale or missing quote, a closed browser, a stopped app, or a provider delay c
an prevent timely detection. No stop order is sent to a broker.
SQLite schema
Table
Role
Notable fields
holdings
Current position state
id, symbol, quantity, buy_price, stop_loss, status, created_at, last_price
trades
Append-style buy/exit event history
event_key (unique), holding_id, timestamp, symbol, action, quantity, price, note

app_metadata
App migration marker
key, value
The schema uses a unique event key to avoid duplicate trade events and a holding status guard when recording exits. Data is stored alongside the application, so it is local to this workspace unless separately backed up or moved.

4. Findings, limits, and recommendations

Operational limitations

- Unreliable live data: yfinance is an unofficial data route; quotes may be delayed, throttled, incomplete, or absent.
- Refresh-based stop-loss: detection only occurs when the app reruns and must not be treated as a live order or safeguard.
- Groww integration is separate: API scripts are not wired into the Streamlit dashboard and do not place orders.
- Dependency documentation gap: Groww scripts import python-dotenv and growwapi, which are not declared in requirements.txt.
- Test naming: files beginning with test_ make external account/API calls and are not unit tests suitable for routine test discovery.
- Local single-user persistence: SQLite beside the code is convenient for a demo but has no backup, multi-user, or deployment strategy documented.

Security and privacy hygiene

A local .env file is present. Its contents were not read. Before sharing or publishing the project, ensure that credentials are excluded from version control and that any credentials already exposed are rotated. A root .gitignore was not visible in the inspected project listing; consider ignoring .env, .venv/, and local database files as appropriate.

Recommended next steps

1. Add a root .gitignore for secrets, virtual environments, and local-only data.
2. Clarify whether Groww scripts are supported tools; if yes, document and separate their dependencies and credentials setup.
3. Add unit tests around input validation, buy/exit transactions, stop-loss behavior, and database persistence using mocked quotes.
4. Handle stale / unavailable prices explicitly before updating a holding or evaluating the simulated stop-loss.
5. Separate market-data access and trade persistence from Streamlit UI logic if the app grows, improving testability.
6. Document backup and data-reset procedures for trading_history.db.

5. Verification and conclusion

Checks performed for this report

- Python syntax compilation succeeded for app.py and the three Groww scripts.
- The current virtual environment contains Streamlit, yfinance, pandas, Plotly, dotenv, and growwapi modules.
- The local SQLite database schema was inspected and contains holdings, trades, and app_metadata.
- Groww scripts were not executed because they make external API calls and require account credentials.
- No conventional automated dashboard test suite was identified in the inspected project files.

Conclusion

The project is a functional educational prototype that combines market-data display with local paper trading. Its most valuable qualities are its straightforward workflow, persistent trade history, and explicit disclaimers. Its largest gaps are lack of automated tests, external-data uncertainty, the refresh-dependent stop-loss behavior, and incomplete dependency / secret-handling documentation for the Groww scripts.

It is appropriate as a local learning demo. It should not be represented or used as a production trading system or investment tool without substantial redesign, testing, operational monitoring, secure credential handling, and properly licensed / reliable data and broker integrations.

Report scope: This report describes the visible project files and local database schema. The workspace is not a Git repository, so commit history, branch status, and tracked-file status could not be assessed.
End of report